Jerry Trader Documentation

JerryHong

Started December 2025
Last updated: March 8, 2026

Contents

1 Introduction **3**

2 Frontend Modules **5**
2.1 frontend overview 5
2.2 modules breakdown 6

3 Backend Architecture **6**
3.1 Package Layout 7
3.2 Computation Core (`core/`) 8
3.3 Services 8
3.4 Rust Migration Roadmap 9
3.5 Stage 2.5 — Bars Builder Pipeline 10
3.5.1 Core Design 10
3.5.2 Implementation Phases 11

4 Roadmap **11**

1 Introduction

Summary Jerry Trader is a quant trading platform mainly focus on real-time US pre-market penny stock break out momentum trading strategy, written multi-language (Python + Rust + React). This documentation provides an overview of the app's features, architecture, and usage instructions *based on my trading strategies.*

The project follows a monorepo layout:

- `python` — Python backend (orchestration, I/O, backend services)
- `rust` — Rust computation core (compiled via maturin/PyO3 as `jerry_trader._rust`)
- `frontend` — React + TypeScript frontend (Vite)

Strategy Here is a summary of my strategy based on emperical trading experience, and the modules are designed to better validate, automate this strategy:

- Entry Attention Condition:
  when tickers are getting active, it has a amount of gains suddenly, we do the momentum trading. for my current project, I use Top20 Entry as a detector.
- It's Better If:
  - it has recent hot news with it.
  - the float shares are relatively low, below 20M is preferred.
  - the country location is better not foreign-registered, etc.
  - price is preferred $1–$20
  - the market has all the attention on this one.
  - Order Book liquidity good(intense trade rates with good up-rising momentum)
- It's Better NOT If:
  - Order Book liquidity good but with multiple key-level rejections.
  - price under 0.5
  - big cap companies earnings beat.

It's not saying that if all the conditions fit it will definitly be a good trade. the conditions will be better if all fit, but sometimes there are unknown black swarn ones, so the key metrics will still be price-volume action and orderbook analysis etc.and besides of that, for a long-term profitable trading strategy, also it depends your trading execution and risk management like:

- entry timing and price.
- risk management(marginal account risk etc).
- …

Feature for current version, here are some features that done:

- modules decoupled, driven by redis stream msg.
- rank list column and overview chart for top20 tickers visualization.
- instant news data fetch, process, LLM system prompt sentiment process, persistence.
- stock info data like: float data, basic information fetch, process, persistence.
- ibkr order execution,management,persistence to PostgreSQL.
- real-time bar build and persistence optimized using clickhouse&rust.
- live mode and replay mode support.

2 Frontend Modules

2.1 frontend overview

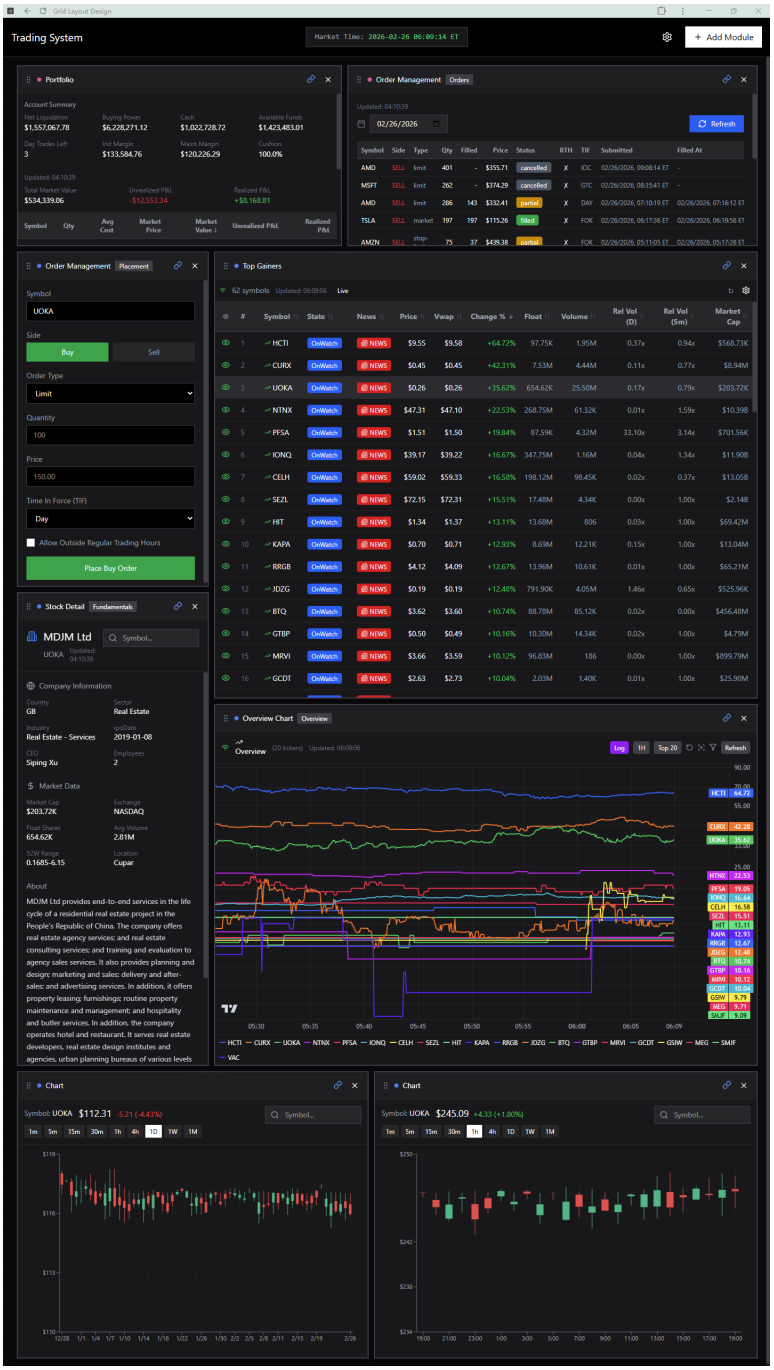

Figure 1: Jerry Trader Application Interface

Platform Overview So, as you can see above, this is my current version of frontend web app interface. but you might wonder, as a rigorous quant trading platform, why would I develop a trading platform that UI is designed for human-centered trade execution?
well, first, I just love trade, my goal is to be profitable quant trader, not researcher due to my lack of rigourous knowlegde of mathematics. second, this project is planned for long-term architecture, I don't have the experience and ability to develop a end-to-end tickdata to machine-decision execution quant platform. the graduate approach is the only choice.

2.2 modules breakdown

Module Type	**Sub-View Name**	**Description**	**Backend**
rank-list		rank list with key metrics	core.snapshot,static-data-worker,news-worker,core.signals
stock-detail	fundamentals	stock basic info	staticdata-worker
stock-detail	news		news-worker
overview-chart	overview		overviewchart-management
overview-chart	focus	focus on one tiker with state segmentation	overviewchart-management
chart			chart-data-service
portfolio		portfolio and positions	OrderManagement
order-management	placement		OrderManagement
order-management	orders		OrderManagement

Table 1: Module descriptions and their corresponding backends.

In the next Chapter, I will break down the details of each module in the above table, which will include:

- data preparation
- data process
- orchestration
- roadmap

3 Backend Architecture

not start to rewind yet. below is just lagacy llm-generated notes&summary

3.1 Package Layout

The Python backend is organized into the following top-level packages:

Package	**Purpose**	**Language**
`core/`	Computation core & Rust migration boundary	Python + Rust
`core/snapshot/`	Snapshot processing: ranks, derived metrics, volume tracking	Python → Rust
`core/factors/`	Factor engine: trade_rate, accel, z-scores	Python → Rust
`core/signals/`	State/signal detection (QUIET ↔ ACTIVE)	Python
`core/news/`	News classification (LLM-based)	Python
`core/_bridge.py`	Rust ↔ Python dispatch (try _rust, fallback to Python)	Python
BFF	BFF: FastAPI + WebSocket server	Python
`DataManager/`	Data-serving workers & servers (static data, news, tick, chart)	Python
`DataSupply/`	External data sources (Polygon, ThetaData, local files)	Python
`OrderManagement/`	IBKR order placement, portfolio, persistence	Python
`MachineLearning/`	ML experimentation (LightGBM)	Python
`utils/`	Cross-cutting: config, logger, Redis keys, session	Python

Table 2: Python backend package layout.

3.2 Computation Core (`core/`)

The `core/` package is the **Rust migration boundary**. Each subpackage follows a compute/orchestration split:

- `compute.py` — Pure functions & stateful trackers (**Rust migration target**). These have planned counterparts in `rust/src/`.
- `processor.py` / `engine.py` — I/O orchestration (Redis, InfluxDB, threads). Stays in Python.

The `_bridge.py` module provides a try/except dispatch:

```
try:
    from jerry_trader._rust import VolumeTracker,
    ↪  compute_ranks
except ImportError:
    from jerry_trader.core.snapshot.compute import
    ↪  VolumeTracker, compute_ranks
```

This allows incremental migration — Rust implementation when compiled, Python fallback otherwise.

3.3 Services

- **BackendForFrontend (BFF)**
  FastAPI + WebSocket server running on the main machine. Collects and preprocesses (`core.snapshot`); receives and emits rank data and overview-chart data to the frontend.
- **core.snapshot (SnapshotProcessor)**
  Receives raw market snapshots from Redis stream, computes ranks and derived metrics (via Rust when available), manages the top-movers subscription set, writes to output Redis stream and InfluxDB.
- **core.factors (FactorManager)**
  Real-time factor engine. Ingests tick-level trade data. Computes trade rate. Computes acceleration. Produces rolling z-scores. Outputs to InfluxDB. Uses Redis (HSET) for metadata. Includes historical bootstrap via Polygon API.
- **core.signals (StateEngine)**
  Polls factor HSETs and fires state transition events (QUIET → ACTIVE, ACTIVE → QUIET) based on z-score thresholds.
- **core.news (NewsProcessor)**
  LLM-based news classification. Determines if news articles are positive catalysts using DeepSeek API.
- **OrderManagement**
  Order management system based on Interactive Brokers API. Handles order placement, status callbacks, portfolio and position tracking. Backed by PostgreSQL via SQLAlchemy + Alembic.

- **DataSupply**
  External data sources:
  - `snapshotDataSupply` — market snapshot collection & replay
  - `tickDataSupply` — real-time tick data (Polygon.io, ThetaData, local Rust replay)
  - `staticdataSupply` — fundamentals, borrow fee, news fetch
  - `bootstrapdataSupply` — historical data loaders
- **DataManager**
  Data-serving workers that bridge DataSupply to the BFF:
  - `static_data_worker` — fetches fundamentals for subscribed tickers
  - `news_worker` — fetches and caches news
  - `tickdata_server` — serves tick data to frontend and factor engine
  - `chart_data_service` — historical bar data for TradingView chart
- **utils**
  Cross-cutting utilities: config, logger, Redis key naming, session management, path resolution.

3.4 Rust Migration Roadmap

The migration from Python `compute.py` modules to Rust follows a three-step progression, ordered from simplest to most complex:

1. **Step 1: Pure math functions** (`core/factors/compute.py`)
   `z_score` and `price_accel` — stateless, no external dependencies. This step establishes the full end-to-end workflow: write Rust in `lib.rs`, run `maturin develop`, `_bridge.py` auto-switches, existing tests validate correctness, `RUST_FACTORS` flips to `True`.

2. **Step 2: Stateful tracker** (`core/snapshot/compute.py`)
   `VolumeTracker` as a `#[pyclass]` with methods — learn stateful Rust objects exposed to Python via PyO3.

3. **Step 3: DataFrame operations** (`core/snapshot/compute.py`)
   `compute_ranks`, `compute_derived_metrics`, `compute_weighted_mid_price` — Polars DataFrame in/out, requires the `pyo3-polars` crate. Most complex.

Note: `core/signals/` (StateEngine) is excluded from the Rust migration roadmap. The signal/state detection logic depends on strategy rules that are not fully decided yet; it will remain Python-only for the foreseeable future.
Status: Step 1 completed. `z_score` achieved 3.6–10.7× speedup, `price_accel` achieved 8.9–12.1× speedup over Python. Steps 2–3 are deferred in favour of the Bars Builder pipeline below.

3.5 Stage 2.5 — Bars Builder Pipeline

The Chart module requires a reliable server-side bar-building pipeline. The v1.0 architecture (frontend fetches historical bars from Polygon API, real-time last-bar-only update from raw trade ticks in `chartDataStore.ts`) suffered from:

- Bar consistency issues across timeframes
- Duplicated aggregation logic (frontend *and* factor engine)
- No single source of truth for completed bars
- Cannot scale to feed downstream engines (factors, signals, risk)

The v2.0 architecture introduces a **Rust-native Bar Builder** as the canonical bar source for all downstream consumers:

```
Polygon / ThetaData / Replayer ticks
         |
         v
  UnifiedTickManager (Python, existing)
         |  asyncio.Queue
         v
  +----------------------------+
  |  BarBuilder (Rust #[pyclass])  |
  |  * ingest_trade()          |
  |  * get_current_bar()       |
  |  * completed-bar callbacks |
  +-------------+--------------+
               |
     +----------+----------+----------+
     |          |          |          |
     v          v          v          v
 ClickHouse  Frontend  FactorEngine  Future:
 (persist)   (WS push)  (bars)     signals, risk
```

3.5.1 Core Design

Timeframes. Eight concurrent timeframes per ticker: 10 s, 1 m, 5 m, 15 m, 1 h, 4 h, 1 d, 1 w. Each is maintained as an independent rolling `BarState` inside the Rust `BarBuilder`.

Session awareness. Bars respect US market session boundaries:

- Premarket: 04:00–09:30 ET
- Regular: 09:30–16:00 ET
- Afterhours: 16:00–20:00 ET

A bar never spans a session boundary. For example, a 5 m bar starting at 09:28 closes at 09:30 (truncated), and a new bar begins at 09:30 in the regular session.

BarState fields. Each rolling bar tracks: open, high, low, close, volume, trade_count, vwap (via cumulative numerator + denominator), bar_start (epoch ms), session tag.

Completed bar emission. When a new trade arrives whose timestamp falls outside the current bar window (or crosses a session boundary), the current bar is *completed* and returned to the Python caller. The caller (BarsBuilderService) is responsible for persisting to ClickHouse and pushing to WebSocket subscribers.

3.5.2 Implementation Phases

1. **Phase 1: Rust BarBuilder core**
   `rust/src/bars.rs` — `BarState`, `Timeframe` enum, `SessionCalendar`, `BarBuilder` as `#[pyclass]`. Methods: `ingest_trade()`, `get_current_bar()`, `flush()`. 8 timeframes, session-aware boundaries, VWAP tracking. Exposed via `lib.rs` in the existing `jerry_trader._rust` module.

2. **Phase 2: ClickHouse + Python service**
   ClickHouse schema for OHLCV bars (`ohlcv_bars` table, `ReplacingMergeTree`, partitioned by date). Python `BarsBuilderService`: orchestrates tick ingestion (reuses `UnifiedTickManager`) → Rust `BarBuilder` → ClickHouse writes → WebSocket broadcast of completed bars. ClickHouse replaces InfluxDB for bar storage (InfluxDB retained for snapshots/factors until full migration).

3. **Phase 3: Frontend integration**
   Update `chartDataStore.ts` to receive server-built bars via WebSocket (subscribe) and REST (historical query from ClickHouse). Remove frontend-side `updateFromTrade()` aggregation. TradingView Lightweight Charts `getBars()` / `subscribeBars()` backed by BarsBuilderService.

4. **Phase 4: Downstream consumers**
   FactorEngine consumes completed bars from BarBuilder instead of raw ticks for trade-rate / acceleration computation. Factor output migrated from InfluxDB to ClickHouse. Factor visualization overlay in the Chart module. Foundation for v3.0 stream bus: bar builder, feature engine, strategy engine as parallel consumers off a shared tick bus.

4 Roadmap

-
- Break out Context build, model based.